

졸업작품 보고서

# bloom

물리 키링과 연동되는 습관 트래킹 iOS 앱

2026 년 5 월 27 일

32210046 무역학과 강민지

## 목차

1. 프로젝트 개요
2. 개발 배경 및 목적
3. 시스템 설계
  - 3.1 전체 아키텍처
  - 3.2 데이터 모델
  - 3.3 화면 구성
4. 핵심 기능 구현
  - 4.1 꽃 시각화 (StoryFlowerView)
  - 4.2 커스텀 레이아웃 (FlowLayout)
  - 4.3 디자인 시스템 (BloomTheme)
  - 4.4 아이콘 마이그레이션 로직
  - 4.5 통계 및 분석 기능
5. 기술 스택
6. 구현 결과
  - 6.1 구현된 기능 목록
  - 6.2 프로젝트 구조
  - 6.3 화면 구현 결과
7. 향후 개발 계획
8. 결론

## 1. 프로젝트 개요

bloom 은 물리적인 키링 버튼 하나를 누르면 할 일이 체크되는 습관 관리 iOS 네이티브 앱이다.

스마트폰 투두 앱은 앱을 열어야 한다는 진입 장벽이 존재하여 습관 형성에 불리하다는 문제의식에서 출발하였다. bloom 은 항상 손에 들고 다니는 키링에 버튼을 달아, 물리적 인터랙션으로 할 일 체크의 마찰을 최소화하는 것을 핵심 목표로 삼는다.

매일 할 일을 완료할수록 꽃잎이 하나씩 피어나는 시각적 피드백을 통해 사용자가 루틴을 꾸준히 유지할 수 있도록 동기를 부여한다.

| 항목       | 내용                                 |
|----------|------------------------------------|
| 플랫폼      | iOS 네이티브 (SwiftUI)                 |
| 개발 언어    | Swift 5.9                          |
| UI 프레임워크 | SwiftUI                            |
| 데이터 저장   | UserDefaults (온디바이스)               |
| 외부 의존성   | 없음 (순수 Swift / Apple SDK)          |
| 타겟 사용자   | 루틴 관리에 관심 있는 20 대 (Z 세대 별다꾸 문화 반영) |

## 2. 개발 배경 및 목적

### 2.1 문제 정의

현대인의 루틴 관리 실패 원인을 분석하면, 기존 투두 앱의 공통된 진입 장벽을 발견할 수 있다.

앱을 실행하고, 화면을 열고, 해당 항목을 찾아 탭하는 일련의 과정이 습관 형성에 필요한 '즉각성'을 저해한다. 특히 아침 운동, 물 마시기, 비타민 복용과 같이 신체적 행동과 동반되는 루틴의 경우, 스마트폰 조작이라는 추가 행동이 오히려 루틴의 흐름을 끊는 요인이 된다.

### 2.2 해결 접근

bloom 은 이 문제를 '물리적 인터랙션'으로 해결한다. 가방이나 지갑에 항상 달고 다니는 키링에 BLE(Bluetooth Low Energy) 기반의 버튼 7 개를 장착하고, 각 버튼에 특정 루틴을 매핑한다.

사용자는 스마트폰을 꺼내지 않고도 키링 버튼 하나를 누르는 것만으로 해당 루틴을 완료 처리할 수 있다.

### 2.3 디자인 컨셉

Z 세대의 '별다꾸(별걸 다 꾸민다)' 문화를 반영한 감성적 비주얼을 핵심 디자인 방향으로 채택하였다.

루틴 완료 시 꽃잎이 피어나는 시각적 메타포를 통해 단순한 체크리스트를 넘어, 매일 성장하는 경험을 제공한다.

## 3. 시스템 설계

### 3.1 전체 아키텍처

본 앱은 SwiftUI 의 데이터 흐름 패턴을 준수하는 단방향 MVVM 구조로 설계되었다.

| 레이어       | 구성 요소                                  | 역할                     |
|-----------|----------------------------------------|------------------------|
| View      | HomeView, MapView, LogView, StatView 등 | UI 렌더링 및 사용자 인터랙션 처리   |
| ViewModel | AppData (ObservableObject)             | 상태 관리, 비즈니스 로직, 영속성 처리 |
| Model     | Mapping (Codable), [String:[Int]]      | 도메인 데이터 구조 정의          |
| Theme     | BloomColor, BloomFont, BloomRadius     | 디자인 토큰 중앙화             |

AppData 는 @MainActor 로 선언된 ObservableObject 로, mappings 와 logs 두 개의 @Published 프로퍼티를 중심으로 앱 전체 상태를 관리한다. 각 프로퍼티에는 didSet 을 통한 자동 영속화 로직이 내장되어 있어, 별도의 저장 호출 없이 상태 변경 즉시 UserDefaults 에 반영된다.

### 3.2 데이터 모델

#### Mapping

키링의 각 버튼과 루틴을 연결하는 핵심 모델이다. Codable, Identifiable, Equatable 을 채택하여 UserDefaults 직렬화 및 SwiftUI 의 ForEach 활용이 가능하다.

| 프로퍼티     | 타입     | 설명                             |
|----------|--------|--------------------------------|
| id       | Int    | 버튼 번호 (0~6), 키링의 물리 버튼과 1:1 매핑 |
| name     | String | 루틴 이름 (빈 문자열 시 '미설정' 처리)       |
| icon     | String | SF Symbol 이름 (15 종 제공)         |
| colorHex | String | 색상 헥스 코드 (8 종 팔레트)             |

## 로그 (logs)

[String: [Int]] 타입으로, 날짜 문자열(yyyy-MM-dd)을 키로, 해당 날짜에 완료된 루틴 ID 배열을 값으로 저장한다.

## 3.3 화면 구성

| 화면      | 파일                      | 주요 기능                        |
|---------|-------------------------|------------------------------|
| 홈 (스토리) | HomeView.swift          | 꽃 시각화, 꽃잎 탭 토글, 진행률 표시       |
| 매핑      | MapView.swift           | 7 개 버튼 루틴 설정, 아이콘·색상 선택      |
| 기록      | LogView.swift           | 30 일 닷 그리드, 연속 스트릭, 오늘 완료 현황 |
| 통계      | StatView.swift          | 주간 바 차트, 월간 달성률, 루틴 순위       |
| 온보딩     | OnboardingView.swift    | 최초 실행 시 4 슬라이드 튜토리얼          |
| 주간 리뷰   | WeeklyReviewSheet.swift | 7 일 꽃 시각화, 베스트 루틴, 달성 메시지    |

## 4. 핵심 기능 구현

### 4.1 꽃 시각화 (StoryFlowerView)

bloom 의 핵심 UI 컴포넌트이다. 매핑된 루틴 수에 따라 꽃잎 수가 동적으로 결정되며, 각 꽃잎은 해당 루틴의 완료 여부에 따라 시각적 상태가 변한다. 꽃잎 배치 알고리즘은 중심점으로부터 orbit 반지름만큼 떨어진 위치에, 전체 루틴 수( $n$ )를 360 도로 균등 분할한 각도( $360/n$  도)로 꽃잎을 배치한다.

- 완료된 꽃잎: 따뜻한 황금색 RadialGradient + 발광 효과 (shadow)
- 미완료 꽃잎: 크림색 그라디언트 + 얇은 테두리
- 중앙 원: 한 개 이상 완료 시 황금색으로 전환, 완료 카운터 표시
- 탭 인터랙션: `contentShape(Ellipse())`로 꽃잎 외곽선에 맞는 정확한 탭 영역 지정

### 4.2 커스텀 레이아웃 (FlowLayout)

아이콘 선택 피커(MapView)의 격자 배치를 위해 SwiftUI 의 Layout 프로토콜을 직접 구현하였다. iOS 16 에서 공식 도입된 이 프로토콜을 활용하여, 사용 가능한 너비에 맞게 아이콘이 자동 줄바꿈되는 플로우 레이아웃을 구현하였다.

### 4.3 디자인 시스템 (BloomTheme)

- BloomColor: 크림 계열 배경색, 잉크 계열 텍스트색, 골드 포인트색 등 18 종 시맨틱 색상 토큰
- BloomFont: DM Serif Display(제목), Noto Sans KR(본문) 두 종의 커스텀 폰트 통합 관리
- BloomCardStyle: ViewModifier 로 구현한 카드 스타일 (.bloomCard() 확장 메서드)

### 4.4 아이콘 마이그레이션 로직

이모지에서 SF Symbol 로 전환 시 기존 사용자 데이터 하위 호환성을 보장하기 위해 `migrateIcons()` 정적 메서드를 구현하였다. 이모지-SF Symbol 매핑 딕셔너리를 통해 앱 업데이트 시 자동 변환한다.

## 4.5 통계 및 분석 기능

- `streakCount()`: 최근 60 일 역순 순회로 연속 달성 일수 산출
- `weekData()`: 현재 주 월~일 각 요일별 달성률, 완료 수, 퍼펙트 데이 수 반환
- `routineCounts(in:)`: 특정 날짜 집합 내 루틴별 누적 완료 횟수 산출

## 5. 기술 스택

| 분류       | 기술                              | 활용 내용                                      |
|----------|---------------------------------|--------------------------------------------|
| 언어       | Swift 5.9                       | 전체 앱 개발                                    |
| UI 프레임워크 | SwiftUI                         | 선언형 UI, 애니메이션, 커스텀 레이아웃                    |
| 상태 관리    | Combine (ObservableObject)      | 반응형 상태 전파 (@Published, @EnvironmentObject) |
| 데이터 영속화  | Foundation (UserDefaults)       | Codable 기반 JSON 직렬화/역직렬화                   |
| 아이콘      | SF Symbols 5                    | 15 종 커스텀 아이콘 팔레트                           |
| 폰트       | DM Serif Display / Noto Sans KR | Google Fonts 기반 커스텀 타이포그래피                 |
| 빌드 도구    | Xcode 15                        | 프로젝트 빌드, 시뮬레이터 테스트                         |
| 버전 관리    | Git                             | 소스 코드 형상 관리                                |

## 6. 구현 결과

### 6.1 구현된 기능 목록

- 홈 화면: 루틴 수에 따라 동적으로 변하는 꽃 시각화, 꽃잎 탭으로 루틴 완료/취소 토글, 실시간 진행률 바·연속 달성 일수·완료 카운터 표시
- 매핑 화면: 키링 7개 버튼 각각에 루틴 이름·SF Symbol 아이콘·색상 지정, 인라인 편집 패널
- 기록 화면: 최근 30일 달성을 닷 그리드, 연속 달성 스트릭, 오늘 완료 현황 리스트, 하루 마무리/초기화 기능
- 통계 화면: 이번 주 요일별 달성을 바 차트, 이번 달(30일) 누적 달성을, 루틴별 달성 순위
- 주간 리뷰 시트: 7일 꽃잎 시각화, 주간 달성을/완벽한 날/총 완료 통계, 베스트 루틴, 맞춤 응원 메시지
- 온보딩: 최초 실행 시 4 슬라이드 인터랙티브 튜토리얼

### 6.2 프로젝트 구조

| 디렉토리/파일                                | 설명                                  |
|----------------------------------------|-------------------------------------|
| BloomApp.swift                         | 앱 진입점 (@main), 라이트 모드 고정            |
| Models/AppData.swift                   | 전체 상태 관리, 비즈니스 로직, UserDefaults 영속화 |
| Models/Mapping.swift                   | 루틴-버튼 매핑 도메인 모델                     |
| Theme/BloomTheme.swift                 | 색상·폰트·반경 토큰, ViewModifier, 상수 정의    |
| Views/ContentView.swift                | 탭 네비게이션, 온보딩·리뷰 오버레이 관리             |
| Views/HomeView.swift                   | 스토리 홈 화면, 배경 그라디언트                  |
| Views/MapView.swift                    | 매핑 편집 UI, FlowLayout 활용             |
| Views/LogView.swift                    | 기록·닷그리드·스트릭 화면                      |
| Views/StatView.swift                   | 통계 차트·순위 화면                         |
| Views/OnboardingView.swift             | 온보딩 슬라이드, 커스텀 아트웍                   |
| Views/WeeklyReviewSheet.swift          | 바텀 시트 형태의 주간 리뷰                     |
| Views/Components/StoryFlowerView.swift | 핵심 꽃 시각화 컴포넌트                       |

### 6.3 화면 구현 결과

아래 스크린샷은 실제 기기 시연 영상에서 캡처한 화면으로, 사용자의 앱 최초 실행부터 통계 확인까지의 전체 흐름을 순서대로 보여준다.



그림 1. 온보딩 1 슬라이드 — 앱 소개



그림 2. 온보딩 2 슬라이드 — 키링 컨셉



그림 3. 온보딩 3 슬라이드 — 버튼-루틴 매핑 설명



그림 4. 온보딩 마지막 슬라이드 — 시작하기



그림 5. 매핑 편집 — 이름·아이콘·색상 설정

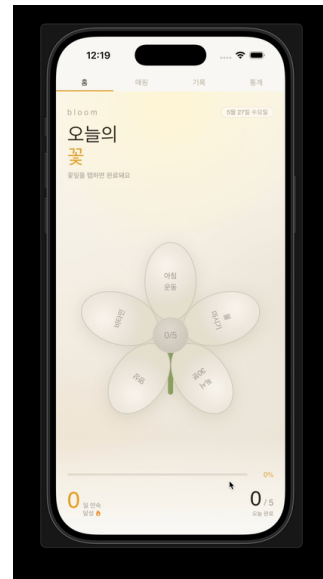


그림 6. 홈 화면 — 루틴 시작 전 (0%)



그림 7. 홈 화면 — 꽃잎 1 개 완료 (17%)

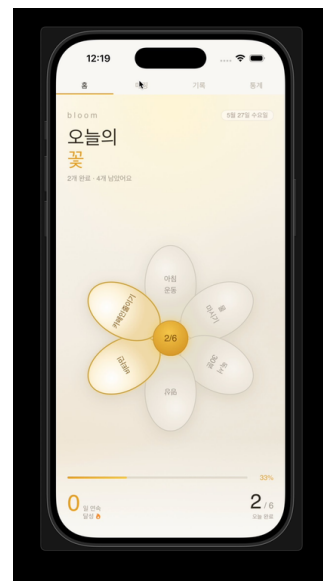


그림 8. 홈 화면 — 꽃잎 2 개 완료 (33%)

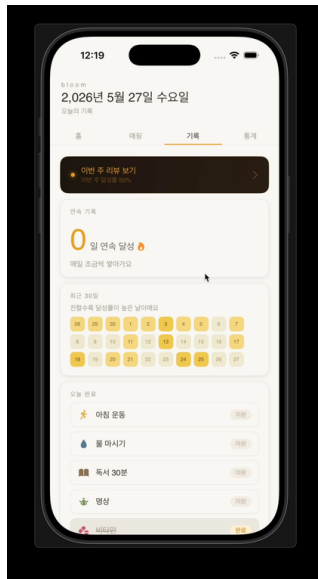


그림 9. 기록 화면 — 30 일 닷 그리드·스트릭

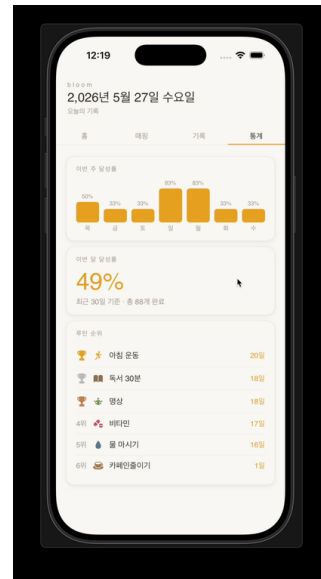


그림 10. 통계 화면 — 바 차트·루틴 순위



그림 11. 기록 화면 — 주간 리뷰 시트 (달성률·베스트 루틴·요일별 꽃잎)

## 7. 향후 개발 계획

### 7.1 BLE 키링 하드웨어 연동

본 프로젝트의 궁극적 목표인 물리 키링과의 연동을 구현한다. ESP32 또는 Arduino 기반의 BLE HID 장치를 제작하고, iOS의 Core Bluetooth 프레임워크를 통해 버튼 입력을 수신하는 방식으로 구현할 예정이다.

### 7.2 클라우드 동기화 및 멀티 디바이스 지원

현재 UserDefaults 기반의 온디바이스 저장 방식을 Supabase(BaaS) 연동으로 확장하여, 여러 기기 간 데이터 동기화를 지원한다.

### 7.3 위젯 및 알림

WidgetKit을 활용한 홈 화면 위젯을 추가하여, 앱을 열지 않고도 오늘의 달성률을 확인할 수 있는 환경을 제공한다. 또한 UNUserNotificationCenter를 통한 루틴 리마인더 기능을 구현한다.

### 7.4 크라우드펀딩

제품 완성도가 충분히 확보되면 텀블벅 등의 크라우드펀딩 플랫폼을 통해 실물 키링 디바이스 양산을 추진한다.

## 8. 결론

본 졸업작품 bloom 은 '습관 관리의 마찰을 최소화한다'는 명확한 문제의식으로부터 출발한 iOS 네이티브 애플리케이션이다. SwiftUI 의 선언형 UI 패러다임을 충실히 따르는 MVVM 아키텍처 위에, 꽃 시각화라는 독창적인 UI 컴포넌트와 물리 키링 연동이라는 하드웨어-소프트웨어 통합 비전을 구현하였다.

외부 라이브러리 없이 순수 Apple SDK 만으로 Custom Layout 프로토콜, Codable 기반 데이터 모델, RadialGradient 기반 시각화, ViewModifier 패턴을 통한 디자인 시스템 등 SwiftUI 의 다양한 고급 기능을 직접 구현한 점에서 기술적 완성도를 높였다.

물리 버튼의 즉각성과 꽃 피우기의 시각적 성취감을 결합한 bloom 은 Z 세대 타겟의 감성적 디자인과 실용적 기능성이 균형을 이루는 제품으로, 향후 BLE 하드웨어 연동 및 클라우드 동기화가 완성되면 실제 서비스로 출시 가능한 수준의 완성도를 목표로 한다.